

CHATBOT PARA DIAN SISTEMAS

DOCUMENTO DE DISEÑO
REALIZADO POR VÍCTOR HERRERO Y ADRIÁN SALAS

CONTENIDO

Descripción general	3
Objetivos básicos.....	3
Tecnologías.....	3
Python	3
Django	3
Bootstrap5	3
spaCy	4
Ollama y Llama3.2.....	4
Almacenaje de datos	4
Arquitectura general	4
Aplicaciones del proyecto	6
Core.....	6
Chat.....	6
Entrenamiento	6
Notificaciones	6
Clasificación de mensajes con spaCy.....	6
Seguridad.....	6
Línea de tiempo.....	7
Próximos pasos posibles.....	7
Glosario	8

DESCRIPCIÓN GENERAL

Este proyecto es una aplicación web construida con Django que implementa un chatbot en forma de widget reutilizable. El objetivo es que Dian Sistemas pueda incrustarlo en su propia página web mediante un iframe, de forma que sus visitantes puedan resolver dudas y ser derivados al departamento adecuado sin salir de la web corporativa.

El widget aparece como un botón flotante en la esquina inferior derecha de la página; al pulsarlo, se despliega con una animación la ventana de conversación. Todo el procesamiento del lenguaje, el almacenamiento de la conversación y el envío de notificaciones ocurre en nuestro servidor Django, de forma transparente para la web del cliente.

OBJETIVOS BÁSICOS

Crear un widget reutilizable para que se pueda incrustar sobre la página de Dian Sistemas. Este chatbot debe ser capaz de:

- Mantener una conversación con el cliente, entendiendo preguntas formuladas en lenguaje natural.
- Clasificar la conversación como consulta técnica o compra, para poder priorizarla y enrutarla correctamente.
- Generar un resumen del chat para mandarlo al departamento correspondiente, evitando que el equipo tenga que leer la conversación completa.

TECNOLOGÍAS

PYTHON

Lenguaje de programación utilizado como base de todo el proyecto. Se ha elegido por su sintaxis legible, por ser el lenguaje nativo tanto de Django como de spaCy.

DJANGO

Framework web de alto nivel escrito en Python, utilizado como base del backend del proyecto. Sigue el patrón Modelo-Vista-Plantilla (MVT), que separa la gestión de datos (modelos), la lógica de negocio (vistas) y la generación del HTML (plantillas).

BOOTSTRAP5

Framework CSS utilizado para la maquetación del widget y de las vistas de administración internas.

En este proyecto interesa especialmente porque permite montar rápidamente componentes como el botón flotante, la ventana de chat, los formularios y las burbujas de mensaje con un aspecto responsive y cuidado sin escribir CSS desde cero.

SPACY

Librería de procesamiento de lenguaje natural (NLP) en Python, orientada a un uso en producción más que a la investigación. Procesa el texto mediante un pipeline de componentes encadenados (tokenizador, etiquetador gramatical, reconocedor de entidades, clasificador de texto, etc.) y admite modelos preentrenados para varios idiomas, incluido el español.

En el chatbot se usan principalmente dos capacidades de spaCy: el componente de clasificación de texto (textcat), para decidir si un mensaje corresponde a una consulta técnica o a una compra, y el reconocimiento de entidades (NER), para detectar en el mensaje elementos relevantes como el nombre de un servicio o producto.

OLLAMA Y LLAMA3.2

Ollama es la herramienta que se usa para ejecutar en local el modelo de lenguaje Llama3.2, exponiéndolo mediante una API que el backend Django consulta para generar las respuestas del chatbot en lenguaje natural.

En vez de un único modelo genérico, el proyecto usa tres modelos separados, cada uno especializado en un tipo de respuesta; en función de la intención detectada por spaCy (consulta técnica, compra u otra), se selecciona el modelo de Ollama adecuado para generar la respuesta al usuario, lo que permite respuestas más ajustadas a cada caso que un único prompt genérico.

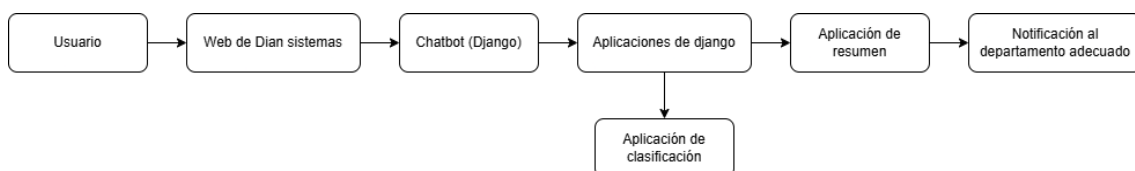
ALMACENAJE DE DATOS

De momento vamos a utilizar la base de datos por defecto de Django (SQLite). Se trata de un motor ligero, basado en un único fichero, que no requiere levantar un servidor de base de datos aparte, lo que la hace muy cómoda durante el desarrollo y las primeras pruebas del widget.

SQLite tiene limitaciones de concurrencia que la hacen poco recomendable para un entorno de producción con varias conversaciones simultáneas, por lo que, como ya se recoge en el apartado de próximos pasos, está prevista una migración a un motor como PostgreSQL antes del despliegue definitivo, apoyándose en el sistema de migraciones de Django para no perder los datos ya almacenados.

En la capeta de documentación del proyecto está el diagrama ER que hemos utilizado.

ARQUITECTURA GENERAL



El diagrama resume el flujo de una conversación: el usuario interactúa con el widget incrustado mediante el iframe en la web de Dian Sistemas; las peticiones llegan al backend Django, donde la app Core gestiona la lógica del widget y delega en la app Entrenamiento el procesamiento del mensaje con spaCy; según la intención detectada, se elige uno de los tres modelos de Llama3.2 servidos con Ollama para generar la respuesta; el resultado (intención detectada y respuesta) se guarda a través de la app Chat, y cuando corresponde, la app Notificaciones genera el resumen y lo envía al departamento adecuado.

APLICACIONES DEL PROYECTO

CORE

Aplicación donde está ubicada la parte visual del chatbot y la lógica del widget: la vista que se sirve dentro del iframe, el botón flotante, la animación de apertura y cierre, y la configuración de qué dominios pueden incrustar el widget.

CHAT

Aplicación donde se tratan las conversaciones. Contiene los modelos de mensaje y conversación (remitente, contenido, marca de tiempo, conversación a la que pertenece) y las vistas encargadas de recibir cada mensaje nuevo, guardarlo y devolver la respuesta generada.

ENTRENAMIENTO

Contiene los datos y la lógica de entrenamiento para el chatbot: el corpus de frases de ejemplo por intención, la configuración del pipeline de spaCy, los scripts para reentrenar el modelo cuando se añaden nuevos datos, y el modelo entrenado que usan las demás apps en tiempo real.

NOTIFICACIONES

Apartado donde se resume la información de las conversaciones y se gestiona la lógica del envío de notificaciones: genera el resumen del chat cuando se cierra o se deriva una conversación, y lo envía al departamento correspondiente (técnico o comercial) según la clasificación obtenida.

CLASIFICACIÓN DE MENSAJES CON SPACY

Los mensajes se clasifican en dos categorías principales:

- Compra
- Consulta técnica

Esta clasificación se realiza con el componente textcat del pipeline de spaCy, entrenado con el corpus de la app Entrenamiento. Cada mensaje se evalúa y se le asigna la categoría con mayor probabilidad; cuando la confianza del modelo es baja, la conversación puede marcarse para revisión manual en lugar de clasificarse automáticamente, evitando enrutar mal al cliente. El resultado de la clasificación es el dato principal que usa la app Notificaciones para decidir a qué departamento enviar el resumen de la conversación.

SEGURIDAD

Para controlar qué páginas externas pueden incrustar el widget, la app Core implementa un mixin de Django que se aplica a la vista servida dentro del iframe. Antes de devolver la respuesta, el mixin comprueba el dominio de origen de la petición contra la lista configurada en settings.py mediante la variable DOMINIOS_PERMITIDOS (cargada a su vez desde el .env), y rechaza la petición si el dominio no está autorizado.

LÍNEA DE TIEMPO

- Iframe y configuración básica del proyecto: estructura del proyecto Django, primera versión del botón flotante y la vista incrustable.
- Inicializar el chatbot (introducción de la librería y etiquetas necesarias): instalación de spaCy, definición inicial de intenciones y entidades.
- Sistema de resumen del chat: lógica para condensar una conversación en un resumen legible por el equipo de Dian Sistemas.
- Almacenamiento y optimización del chatbot: ajuste de modelos de datos y del rendimiento del pipeline de spaCy.
- Configuración para mensajería automática: envío automático de notificaciones al departamento correspondiente.

PRÓXIMOS PASOS POSIBLES

- Migración a otro tipo de base de datos, sustituyendo SQLite por un motor apto para producción como MySQL.
- Automatización de detección de servicios ofrecidos por Dian Sistemas, para mantener actualizado el corpus de entrenamiento sin trabajo manual.
- Añadir decoradores de mail para notificaciones con la marca de la empresa, dando un aspecto corporativo a los avisos enviados.
- Métricas en el panel de administración, para poder ver volumen de conversaciones, tasa de clasificación correcta y tiempos de respuesta.
- Comprobar la legislación pertinente al almacenaje de conversaciones, en particular en materia de protección de datos personales.

GLOSARIO

Término	Definición
AJAX	Técnica de JavaScript (Asynchronous JavaScript And XML) que permite enviar y recibir datos del servidor sin recargar la página; se usa en el widget para enviar cada mensaje del usuario y mostrar la respuesta del chatbot en tiempo real.
Llama3.2	Modelo de lenguaje (LLM) de Meta, servido en local mediante Ollama, que el chatbot usa para generar las respuestas en lenguaje natural.
Mixin	Clase reutilizable de Django que añade una comprobación común a varias vistas sin duplicar código; en este proyecto se usa para verificar que el dominio que solicita el iframe está en DOMINIOS_PERMITIDOS antes de servir la vista.
LLM	Modelo de Lenguaje Grande (Large Language Model), un tipo de modelo de inteligencia artificial entrenado con grandes cantidades de texto para comprender y generar lenguaje natural
Chatbot	Programa capaz de mantener una conversación con una persona simulando lenguaje natural, en este caso mediante reglas y modelos entrenados con spaCy.
Corpus	Conjunto de textos de ejemplo (frases de clientes, preguntas frecuentes, etc.) que se usa para entrenar o evaluar el modelo de spaCy.
Entidad (NER)	Elemento concreto que se extrae de un texto (un servicio, una fecha, un nombre de producto). El reconocimiento de entidades nombradas (Named Entity Recognition) es la tarea de spaCy que las identifica.
Iframe	Elemento HTML que permite incrustar una página (en este caso, el chat de Django) dentro de otra página web, la del cliente, manteniendo el contenido aislado.
NLP (Procesamiento del Lenguaje Natural)	Campo de la inteligencia artificial dedicado a que los ordenadores comprendan, interpreten y generen lenguaje humano.
Pipeline (spaCy)	Secuencia de componentes (tokenizador, etiquetador, NER, clasificador de texto...) por los que pasa un texto al ser procesado por spaCy.
postMessage	API de JavaScript que permite que la página del cliente y el contenido del iframe se envíen mensajes entre sí de forma segura, por ejemplo para avisar de que el chat se ha abierto o cerrado.
SQLite	Sistema de base de datos ligero basado en un único fichero, usado por defecto en Django durante el desarrollo por no requerir un servidor de base de datos independiente.
Textcat (clasificador de texto)	Componente de spaCy que asigna una o varias categorías predefinidas a un texto; en este proyecto se usa para distinguir entre consulta técnica y compra.
Token / tokenización	Un token es cada unidad mínima (normalmente una palabra o signo) en la que se divide un texto; la tokenización es el primer paso de cualquier pipeline de spaCy.
Widget	Componente de interfaz reutilizable y autocontenido -en este caso el chatbot- pensado para insertarse dentro de otra página web sin depender de su diseño.
X-Frame-Options	Cabecera HTTP que controla si una página puede mostrarse dentro de un iframe de otro dominio; Django la activa por defecto y hay que exceptuarla en la vista del widget.